Лабораторная работа №1

Подготовка лабораторного стенда

Жукова Арина Александровна

Содержание

1 Цель работы **6**

2 Задание **7**

3 Теоретическое введение **9**

4 Выполнение лабораторной работы **11**

4.1 Создание рабочего каталога курса 11
4.2 Создание рабочего пространства для лабораторной работы (проект DrWatson) 12
4.3 Установка необходимых пакетов 13
4.4 Выполнение базового кода для модели экспоненциального роста 14
4.5 Преобразование кода в литературный стиль 14
4.6 Генерация чистого кода, Jupyter notebook и документации Quarto 15
4.7 Выполнение кода из Jupyter notebook 16
4.8 Интеграция документации Quarto в отчёт 17
4.9 Инициализация проекта и загрузка пакетов 18
4.10 Определение модели 18
4.11 Первый запуск с параметрами по умолчанию 18
4.12 Визуализация результатов 19
4.13 Анализ результатов 19
4.14 Сохранение всех результатов 20
4.15 Параметрическое исследование модели 20
4.16 Генерация производных форматов для параметрической версии 21
4.17 Выполнение Jupyter notebook с параметрами 22
4.18 Интеграция параметрической документации в отчёт 22

5 Параметрическое исследование экспоненциального роста **23**

5.1 Активация проекта и загрузка пакетов 23
5.2 Определение модели 24
5.3 Определение параметров в Dict 24
5.4 Функция-обертка для запуска одного эксперимента 25
5.5 Запуск базового эксперимента 25
5.6 Визуализация базового эксперимента 26
5.7 Параметрическое сканирование 27
5.8 Запуск всех экспериментов и собор результатов 27

5.9 Анализ и визуализация результатов сканирования 29
5.10 Бенчмаркинг с разными параметрами 31
5.11 Сохранение всех результатов 32

6 Выводы **34**

Список литературы **35**

Список иллюстраций

4.1 Создание каталога курса и клонирование шаблона 11
4.2 Результат выполнения make prepare 12
4.3 Инициализация проекта DrWatson 13
4.4 Установка пакетов 13
4.5 Проверка установки 14
4.6 Результат выполнения базового скрипта 14
4.7 Литературный код (фрагмент) 15
4.8 Сгенерированные файлы: чистый код, notebook, Quarto 16
4.9 Выполнение Jupyter notebook 16
4.10 Изменение файла _quarto.yml 17
4.11 Изменение преамбулы 17
4.12 Формирование отчёта 20
4.13 Определение сетки параметров 20
4.14 Результаты параметрического сканирования 21
4.15 Файлы для параметрической версии 21
4.16 Выполнение параметрического notebook 22

Список таблиц

3.1 Основные инструменты, использованные в работе 10

1 Цель работы

Освоить современные приёмы организации вычислительных проектов в области математического моделирования с использованием языка Julia. Научиться применять инструменты литературного программирования (Literate.jl), управления проектами (DrWatson.jl) и генерации отчётов (Quarto) для создания воспроизводимых и хорошо документированных исследований. В качестве содержательного примера реализовать и исследовать модель экспоненциального роста.

2 Задание

1. Создать рабочий каталог для всего курса, следуя соглашению об именовании Denote и структуре, предложенной в методических указаниях.
2. Создать рабочее пространство для программ в рамках лабораторной работы (проект DrWatson внутри каталога `labs/lab01`).
3. Установить необходимые пакеты Julia (DifferentialEquations, Plots, DataFrames, DrWatson, Literate, IJulia, Quarto и др.).
4. Выполнить предложенный код, моделирующий экспоненциальный рост.
5. Преобразовать код в литературный стиль, добавив пояснения на Markdown.
6. Из полученного литературного кода сгенерировать:
   - чистый исполняемый скрипт;
   - Jupyter notebook;
   - документацию в формате Quarto.
7. Выполнить код из Jupyter notebook и убедиться в работоспособности.
8. Интегрировать сгенерированную документацию в отчёт (включить `.qmd`-файлы).
9. Модифицировать литературный код для параметрического исследования (сканирование по параметру скорости роста α).
10. Повторно сгенерировать чистый код, notebook и документацию для версии с параметрами.

11. Выполнить код из нового notebook и проверить результаты.
12. Интегрировать обновлённую документацию в отчёт.

3 Теоретическое введение

Экспоненциальный рост описывается дифференциальным уравнением

$$\frac{du}{dt} = \alpha u, \quad u(0) = u_0,$$

решение которого имеет вид $u(t) = u_0 e^{\alpha t}$. Эта модель применима в биологии (рост популяции в идеальных условиях), финансах (сложные проценты), эпидемиологии (начальная фаза эпидемии) и других областях. Ключевая характеристика — постоянное время удвоения $T_2 = \ln 2/\alpha$.

Литературное программирование (Д. Кнут) предлагает рассматривать программу как связное повествование, где код является неотъемлемой частью документации. В экосистеме Julia эту парадигму поддерживают пакеты:

- `Literate.jl` — преобразование скриптов с Markdown-комментариями в документы различных форматов.
- `Weave.jl` — создание научных отчётов.
- `Jupyter` — интерактивные блокноты.

Для структурирования научных проектов используется `DrWatson.jl`, который задаёт стандартную иерархию каталогов, упрощает сохранение результатов, управление параметрами и воспроизводимость.

Ключевые инструменты, применяемые в работе, сведены в таблицу 3.1.

Таблица 3.1: Основные инструменты, использованные в работе

Пакет / Инструмент	Назначение
`DrWatson.jl`	Организация проекта, управление параметрами, кэширование результатов
`Literate.jl`	Генерация документов из скриптов с Markdown-разметкой
`Quarto`	Создание научных отчётов и презентаций на основе Markdown
`DifferentialEquations.jl`	Численное решение ОДУ
`Plots.jl`	Визуализация данных
`IJulia`	Интеграция Julia с Jupyter Notebook

Более подробно с перечисленными технологиями можно ознакомиться в [**knuth1984**; **schulte2012**; **juliadynamics**].

4 Выполнение лабораторной работы

4.1 Создание рабочего каталога курса

Следуя методическим указаниям, был создан каталог для всего курса в соответствии с соглашением Denote. Для учебного года 2025–2026 (второй семестр) и предмета «Математическое моделирование» (код `mathmod`) структура выглядит следующим образом:

```
mkdir -p ~/work/study/2026-1/2026-1==study--mathmod
cd ~/work/study/2026-1/2026-1==study--mathmod
```

Затем репозиторий был инициализирован с помощью шаблона `course-directory-student-template` (рис. 4.1).

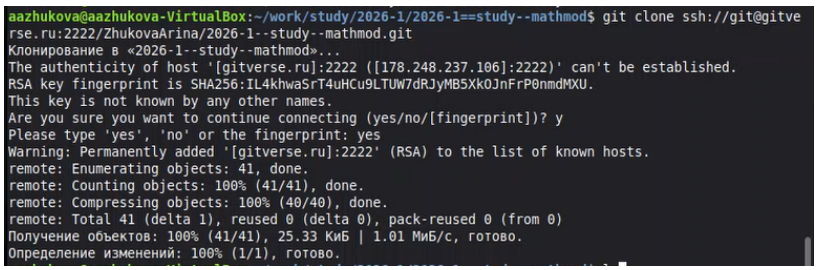


Рисунок 4.1: Создание каталога курса и клонирование шаблона

После клонирования была выполнена настройка курса:

```
echo mathmod > COURSE
make prepare
git add . && git commit -m "feat: init course structure" && git push
```

На рис. 4.2 показан результат выполнения `make prepare`.

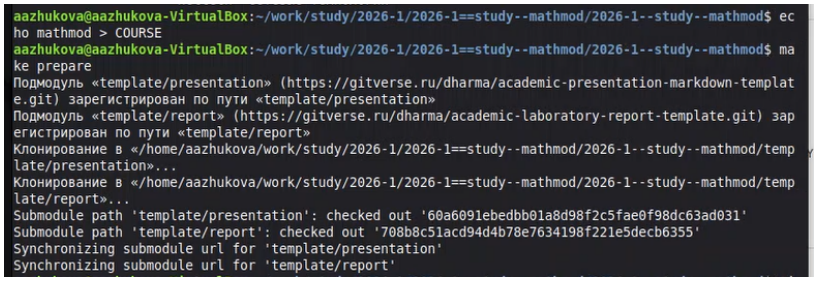


Рисунок 4.2: Результат выполнения make prepare

4.2 Создание рабочего пространства для лабораторной работы (проект DrWatson)

Внутри каталога лабораторных работ `labs` был создан проект DrWatson для первой лабораторной работы:

```
cd labs/lab01
julia
```

В REPL были выполнены команды при помощи скрипта:

```
using Pkg
Pkg.add("DrWatson")
using DrWatson
initialize_project("project"; authors="Жукова Арина", git=false)
cd("project")
```

Скриншот процесса создания проекта приведён на рис. 4.3.

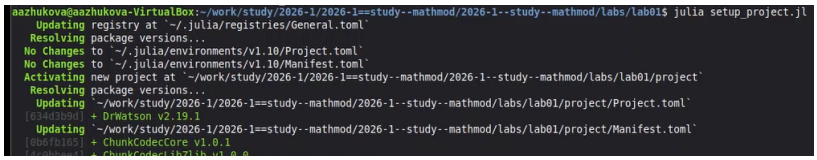


Рисунок 4.3: Инициализация проекта DrWatson

4.3 Установка необходимых пакетов

Для удобства был создан скрипт `add_packages.jl` (листинг ниже), который активирует окружение и устанавливает все требуемые пакеты.

```
#!/usr/bin/env julia
using Pkg
Pkg.activate(".")
packages = [
    "DrWatson", "DifferentialEquations", "Plots", "DataFrames",
    "CSV", "JLD2", "Literate", "IJulia", "BenchmarkTools", "Quarto"
]
Pkg.add(packages)
```

Скрипт был запущен командой `julia add_packages.jl` (рис. 4.4).

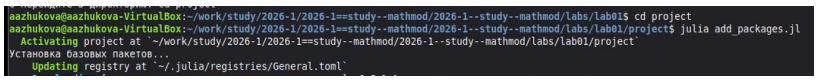


Рисунок 4.4: Установка пакетов

Проверили установки при помощи тестового скрипта

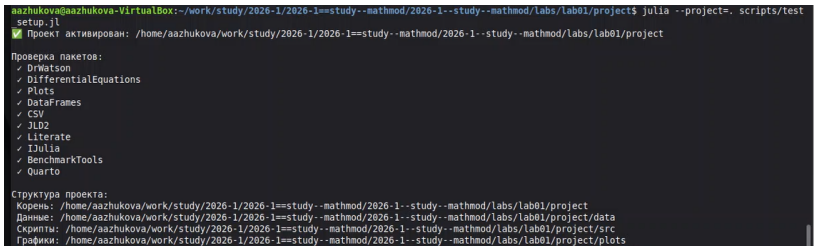


Рисунок 4.5: Проверка установки

4.4 Выполнение базового кода для модели экспоненциального роста

В каталоге `scripts/` был создан файл `01_exponential_growth.jl` с кодом, решающим уравнение $\frac{du}{dt} = \alpha u$ и строящим график. Запуск скрипта:

```
julia --project=. scripts/01_exponential_growth.jl
```

Результат — сгенерированный график в папке `plots/` и вывод первых строк таблицы значений (рис. 4.6).

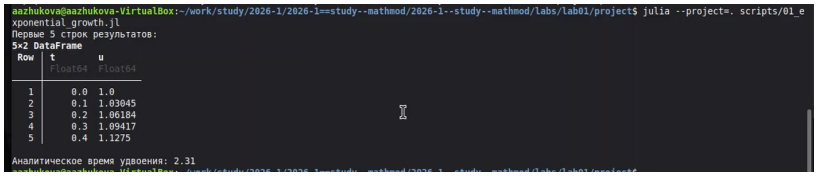
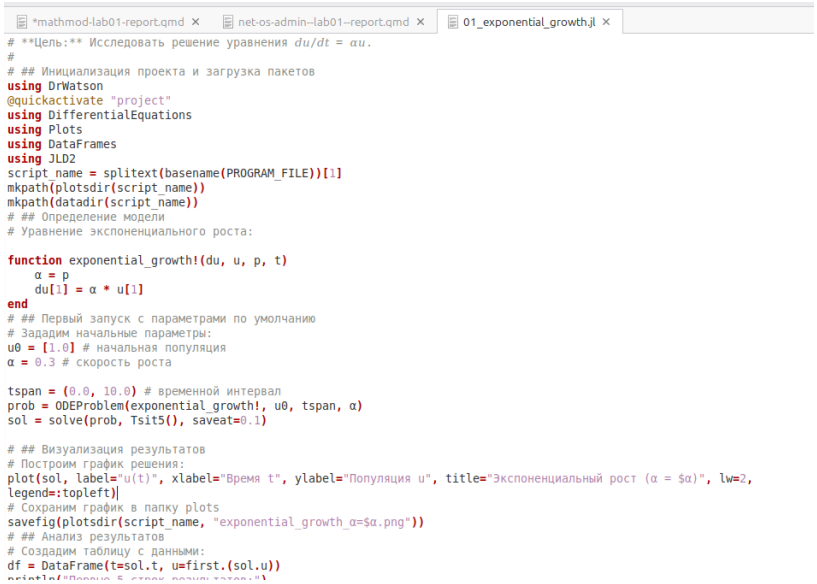


Рисунок 4.6: Результат выполнения базового скрипта

4.5 Преобразование кода в литературный стиль

Исходный скрипт был дополнен Markdown-комментариями, описывающими каждый этап вычислений. Фрагмент литературного кода показан на рис. 4.7.



```
*mathmod-lab01-report.qmd ×   net-os-admin--lab01--report.qmd ×   01_exponential_growth.jl ×
# **Цель:** Исследовать решение уравнения du/dt = αu.
#
# ## Инициализация проекта и загрузка пакетов
using DrWatson
@quickactivate "project"
using DifferentialEquations
using Plots
using DataFrames
using JLD2
script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
# ## Определение модели
# Уравнение экспоненциального роста:

function exponential_growth!(du, u, p, t)
    α = p
    du[1] = α * u[1]
end
# ## Первый запуск с параметрами по умолчанию
# Зададим начальные параметры:
u0 = [1.0] # начальная популяция
α = 0.3 # скорость роста

tspan = (0.0, 10.0) # временной интервал
prob = ODEProblem(exponential_growth!, u0, tspan, α)
sol = solve(prob, Tsit5(), saveat=0.1)

# ## Визуализация результатов
# Построим график решения:
plot(sol, label="u(t)", xlabel="Время t", ylabel="Популяция u", title="Экспоненциальный рост (α = $α)", lw=2,
legend=:topleft)
# Сохраним график в папку plots
savefig(plotsdir(script_name, "exponential_growth_α=$α.png"))
# ## Анализ результатов
# Создадим таблицу с данными:
df = DataFrame(t=sol.t, u=first.(sol.u))
```

Рисунок 4.7: Литературный код (фрагмент)

4.6 Генерация чистого кода, Jupyter notebook и документации Quarto

С помощью скрипта `tangle.jl` (приведён в методичке) были созданы производные форматы:

```
julia --project=. scripts/tangle.jl scripts/01_exponential_growth.jl
```

В результате в каталогах `scripts/01_exponential_growth`, `notebooks/01_exponential_growth` и `markdown/01_exponential_growth` появились соответственно чистый скрипт, Jupyter notebook и документ Quarto (рис. 4.8).

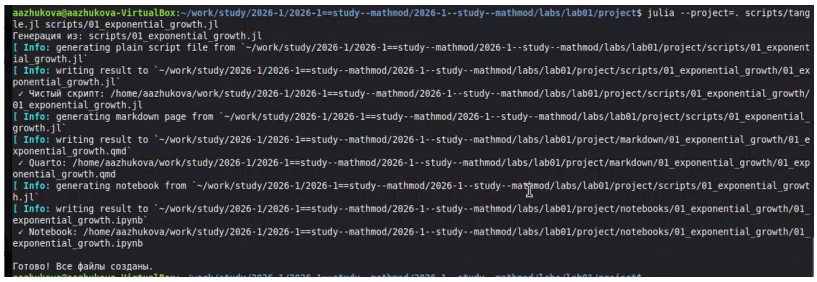


Рисунок 4.8: Сгенерированные файлы: чистый код, notebook, Quarto

4.7 Выполнение кода из Jupyter notebook

Запущен Jupyter notebook `notebooks/01_exponential_growth/01_exponential_growth.ipynb`, все ячейки выполнены успешно (рис. 4.9).

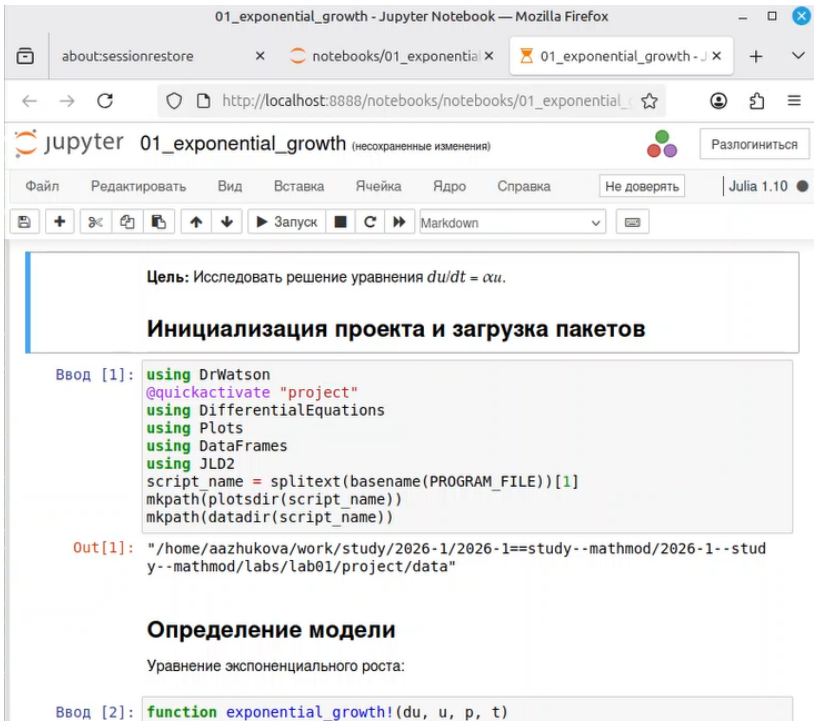


Рисунок 4.9: Выполнение Jupyter notebook

4.8 Интеграция документации Quarto в отчёт

Включили поддержку кода julia в файл _quarto.yml (рис. 4.10).

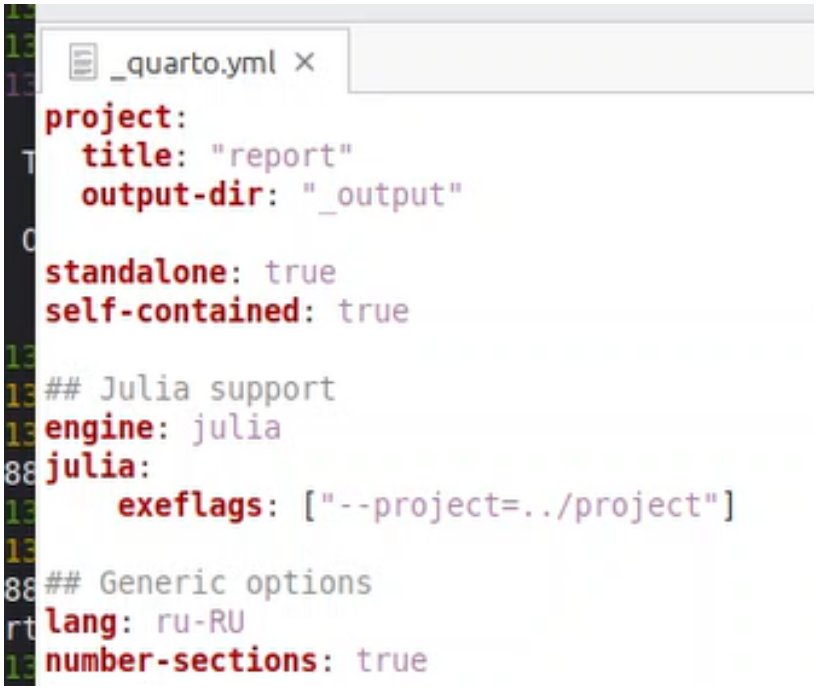


Рисунок 4.10: Изменение файла _quarto.yml

В преамбуле preamble.tex подключили пакет juliamono (рис. 4.11).

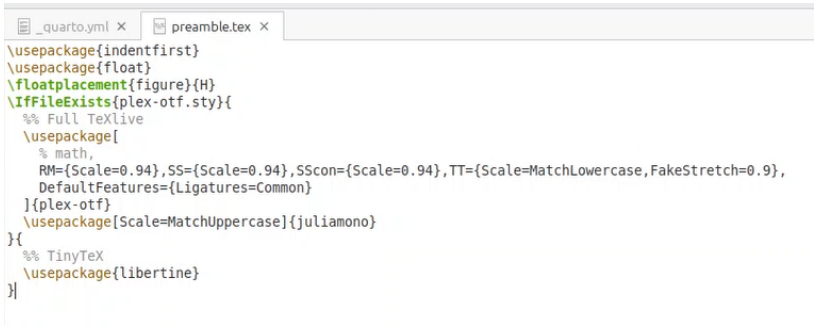


Рисунок 4.11: Изменение преамбулы

Сгенерированный файл `01_exponential_growth.qmd` был включён в настоящий отчёт с помощью директивы `{{< include ... >}}` (см. ниже).

Цель: Исследовать решение уравнения □□/□□ = □□.

4.9 Инициализация проекта и загрузка пакетов

```
using Pkg
Pkg.activate("../project")
using DrWatson
@quickactivate "project"
using DifferentialEquations
using Plots
default(fmt = :png)
using DataFrames
using JLD2

script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

4.10 Определение модели

Уравнение экспоненциального роста:

```
function exponential_growth!(du, u, p, t)
    α = p
    du[1] = α * u[1]
end
```

4.11 Первый запуск с параметрами по умолчанию

Зададим начальные параметры:

```
u0 = [1.0] # начальная популяция
α = 0.3 # скорость роста

tspan = (0.0, 10.0) # временной интервал
prob = ODEProblem(exponential_growth!, u0, tspan, α)
sol = solve(prob, Tsit5(), saveat=0.1)
```

4.12 Визуализация результатов

Построим график решения:

```
plot(sol, label="u(t)", xlabel="Время t", ylabel="Популяция u", title="Экспонен
```

Сохраним график в папку plots

```
savefig(plotsdir(script_name, "exponential_growth_α=$α.png"))
```

4.13 Анализ результатов

Создадим таблицу с данными:

```
df = DataFrame(t=sol.t, u=first.(sol.u))
println("Первые 5 строк результатов:")
println(first(df, 5))
```

Вычислим удвоение популяции:

```
u_final = last(sol.u)[1]
doubling_time = log(2) / α
println("\nАналитическое время удвоения: ", round(doubling_time; digits=2))
```

4.14 Сохранение всех результатов

```
@save datadir(script_name, "all_results.jld2") df
```

Создание pdf файла отчета (рис. 4.12).

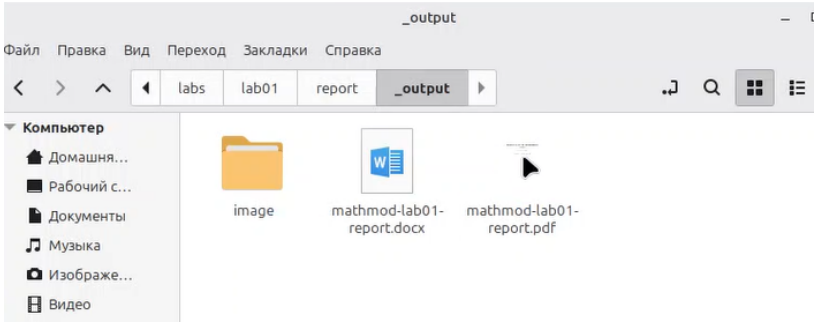


Рисунок 4.12: Формирование отчёта

4.15 Параметрическое исследование модели

Для изучения зависимости решения от параметра α был создан новый литературный скрипт `scripts/02_exponential_growth.jl`. В нём используется функция `produce_or_load` из DrWatson для автоматического кэширования результатов, а также сканирование по сетке значений α. Фрагмент кода с определением сетки параметров приведён на рис. 4.13.

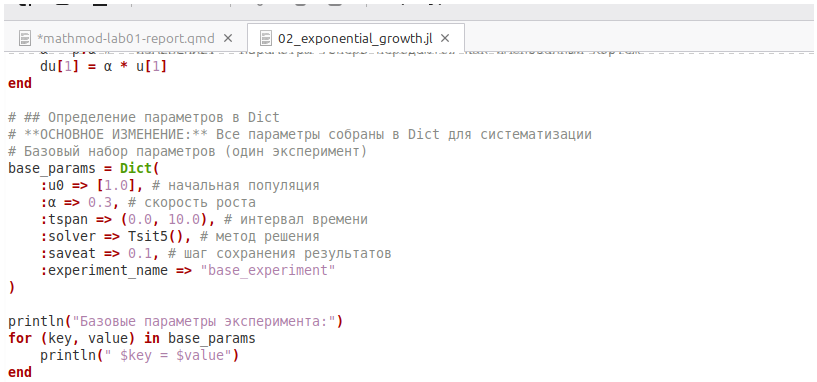


Рисунок 4.13: Определение сетки параметров

После запуска скрипта:

```
julia --project=. scripts/02_exponential_growth.jl
```

были получены графики всех траекторий, зависимость времени удвоения от α и результаты бенчмаркинга (рис. 4.14).

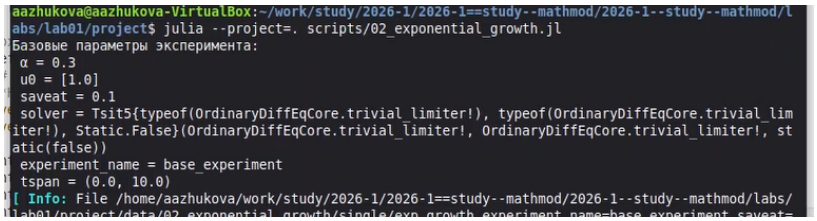


Рисунок 4.14: Результаты параметрического сканирования

4.16 Генерация производных форматов для параметрической версии

Аналогично предыдущему шагу, с помощью `tangle.jl` были созданы чистый скрипт, Jupyter notebook и Quarto-документ для `02_exponential_growth.jl` (рис. 4.15).

Рисунок 4.15: Файлы для параметрической версии

4.17 Выполнение Jupyter notebook с параметрами

Notebook `notebooks/02_exponential_growth/02_exponential_growth.ipynb` был открыт и выполнен; все ячейки отработали без ошибок (рис. 4.16).

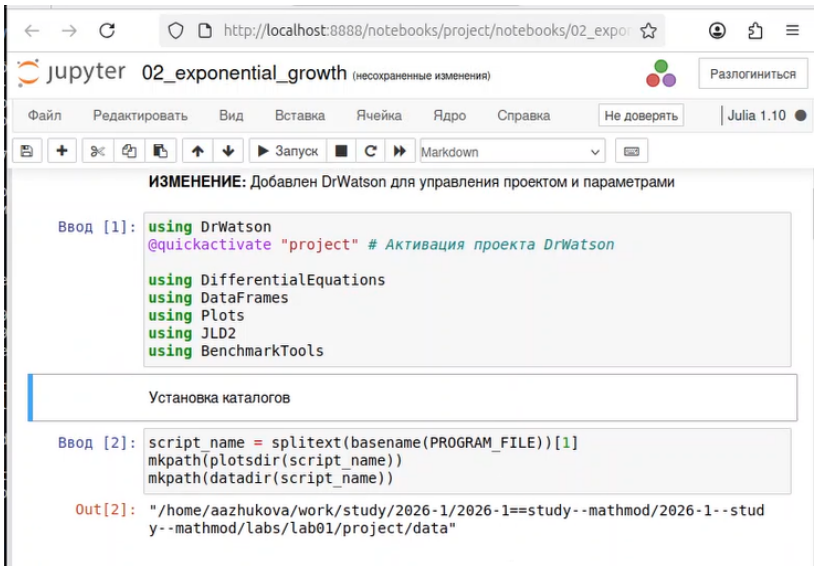


Рисунок 4.16: Выполнение параметрического notebook

4.18 Интеграция параметрической документации в отчёт

Соответствующий файл `02_exponential_growth.qmd` включён ниже.

5 Параметрическое исследование экспоненциального роста

5.1 Активация проекта и загрузка пакетов

ИЗМЕНЕНИЕ: Добавлен DrWatson для управления проектом и параметрами

```
using DrWatson
@quickactivate "project" # Активация проекта DrWatson

using DifferentialEquations
using DataFrames
using Plots
using JLD2
using BenchmarkTools
```

Установка каталогов

```
script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

5.2 Определение модели

Модель: □□/□□ = □ □ □

```
function exponential_growth!(du, u, p, t)
    α = p.α # **ИЗМЕНЕНИЕ:** Параметры теперь передаются как именованный кортеж
    du[1] = α * u[1]
end
```

5.3 Определение параметров в Dict

ОСНОВНОЕ ИЗМЕНЕНИЕ: Все параметры собраны в Dict для систематизации

Базовый набор параметров (один эксперимент)

```
base_params = Dict(
    :u0 => [1.0], # начальная популяция
    :α => 0.3, # скорость роста
    :tspan => (0.0, 10.0), # интервал времени
    :solver => Tsit5(), # метод решения
    :saveat => 0.1, # шаг сохранения результатов
    :experiment_name => "base_experiment"
)

println("Базовые параметры эксперимента:")
for (key, value) in base_params
    println(" $key = $value")
end
```

5.4 Функция-обертка для запуска одного эксперимента

ИСПРАВЛЕНИЕ: Возвращаем Dict со строковыми ключами

```
function run_single_experiment(params::Dict)
    @unpack u0, α, tspan, solver, saveat = params
    prob = ODEProblem(exponential_growth!, u0, tspan, (α=α,)) # Создаем и решае
    sol = solve(prob, solver; saveat=saveat)
    final_population = last(sol.u)[1] # Анализ результатов
    doubling_time = log(2) / α
    return Dict(
        "solution" => sol,
        "time_points" => sol.t,
        "population_values" => first.(sol.u),
        "final_population" => final_population,
        "doubling_time" => doubling_time,
        "parameters" => params # Сохраняем исходные параметры
        ) # Используем строки как ключи для совместимости с DrWatson
end
```

5.5 Запуск базового эксперимента

ИЗМЕНЕНИЕ: Используем produce_or_load для автоматического кэширования

```
data, path = produce_or_load(
    datadir(script_name, "single"), # Папка для сохранения
    base_params, # Параметры эксперимента
    run_single_experiment, # Функция для выполнения
```

```
    prefix = "exp_growth", # Префикс имени файла
    tag = false, # Не добавлять git-тег
    verbose = true
)
println("\nРезультаты базового эксперимента:")
println(" Финальная популяция: ", data["final_population"])
println(" Время удвоения: ", round(data["doubling_time"]; digits=2))
println(" Файл результатов: ", path)
```

5.6 Визуализация базового эксперимента

```
p1 = plot(data["time_points"], data["population_values"],
    label="α = $(base_params[:α])",
    xlabel="Время, t",
    ylabel="Популяция, u(t)",
    title="Экспоненциальный рост (базовый эксперимент)",
    lw=2,
    legend=:topleft,
    grid=true
)
```

Сохраним график в папку plots

```
savefig(plotsdir(script_name, "single_experiment.png"))
```

5.7 Параметрическое сканирование

НОВАЯ СЕКЦИЯ: Исследование влияния параметра α Сетка параметров для сканирования

```
param_grid = Dict(
    :u0 ⇒ [[1.0]], # фиксируем начальное условие
    :α ⇒ [0.1, 0.3, 0.5, 0.8, 1.0], # исследуемые значения скорости роста
    :tspan ⇒ [(0.0, 10.0)], # фиксируем интервал времени
    :solver ⇒ [Tsit5()], # фиксируем метод решения
    :saveat ⇒ [0.1], # фиксируем шаг сохранения
    :experiment_name ⇒ ["parametric_scan"]
)
```

Генерация всех комбинаций параметров

```
all_params = dict_list(param_grid)

println("\n" * "="^60)
println("ПАРАМЕТРИЧЕСКОЕ СКАНИРОВАНИЕ")
println("Всего комбинаций параметров: ", length(all_params))
println("Исследуемые значения α: ", param_grid[:α])
println("="^60)
```

5.8 Запуск всех экспериментов и собор результатов

НОВАЯ СЕКЦИЯ: Автоматический запуск и сохранение всех вариантов

```
all_results = []
all_dfs = []

for (i, params) in enumerate(all_params)
    println("Прогресс: $i/$(length(all_params)) | α = $(params[:α])")
    data, path = produce_or_load(
        datadir(script_name, "parametric_scan"), # Данные
        params, # Текущий набор параметров
        run_single_experiment, # Функция для выполнения
        prefix = "scan", # Префикс имени файла
        tag = false,
        verbose = false # Не выводить подробности для каждого запуска
    ) # Автоматическое сохранение/загрузка каждого эксперимента
    result_summary = merge(
        params,
        Dict(
            :final_population => data["final_population"],
            :doubling_time => data["doubling_time"],
            :filepath => path # Путь к сохраненным данным
        )
    ) # Сохраняем сводные результаты (используем символы для параметров, но дан
    push!(all_results, result_summary)
    df = DataFrame(
        t = data["time_points"],
        u = data["population_values"],
        α = fill(params[:α], length(data["time_points"]))
    ) # Сохраняем полные данные для визуализации
    push!(all_dfs, df)
```

```
end
```

5.9 Анализ и визуализация результатов сканирования

НОВАЯ СЕКЦИЯ: Сравнительный анализ всех экспериментов Сводная таблица результатов

```
results_df = DataFrame(all_results)
println("\nСводная таблица результатов:")
println(results_df[!, [:α, :final_population, :doubling_time]])
```

Сравнительный график всех траекторий

```
p2 = plot(size=(800, 500), dpi=150)

for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment,
        prefix = "scan"
    ) # Загружаем данные (они уже есть на диске)
    plot!(p2, data["time_points"], data["population_values"],
        label="α = $(params[:α])",
        lw=2,
        alpha=0.8
    )
end
plot!(p2,
```

```
    xlabel="Время, t",
    ylabel="Популяция, u(t)",
    title="Параметрическое исследование: влияние α на рост",
    legend=:topleft,
    grid=true
)
```

Сохраним график в папку plots

```
savefig(plotsdir(script_name, "parametric_scan_comparison.png"))
```

График зависимости времени удвоения от α

```
p3 = plot(results_df.α, results_df.doubling_time,
    seriestype=:scatter,
    label="Численное решение",
    xlabel="Скорость роста, α",
    ylabel="Время удвоения, t₂",
    title="Зависимость времени удвоения от α",
    markersize=8,
    markercolor=:red,
    legend=:topright
)
```

Теоретическая кривая: $t_2 = \ln(2)/\alpha$

```
α_range = 0.1:0.01:1.0
plot!(p3, α_range, log(2) ./ α_range,
    label="Теория: t₂ = ln(2)/α",
    lw=2,
```

```
    linestyle=:dash,
    linecolor=:blue
)
```

Сохраним график в папку plots

```
savefig(plotsdir(script_name, "doubling_time_vs_alpha.png"))
```

5.10 Бенчмаркинг с разными параметрами

ИЗМЕНЕНИЕ: Бенчмаркинг для разных значений α

```
println("\n" * "="^60)
println("Бенчмаркинг для разных значений α")
println("="^60)
benchmark_results = []
for α_value in param_grid[:α]
    bench_params = Dict(
    :u0 => [1.0],
    :α => α_value,
    :tspan => (0.0, 10.0),
    :solver => Tsit5(),
    :saveat => 0.1
) # Подготавливаем параметры для бенчмарка
function benchmark_run() # Функция для бенчмарка
    prob = ODEProblem(exponential_growth!,
                      bench_params[:u0],
                      bench_params[:tspan],
                      (α=bench_params[:α],))
```

```
    return solve(prob, bench_params[:solver];
            saveat=bench_params[:saveat])
    end
    println("\nБенчмарк для α = $α_value:")
    b = @benchmark $benchmark_run() samples=100 evals=1 # Запуск бенчмарка
    push!(benchmark_results, (α=α_value, time=median(b).time/1e9)) # время в се
    println(" Среднее время: ", round(median(b).time/1e9; digits=4), " сек")
end
```

График зависимости времени вычисления от α

```
bench_df = DataFrame(benchmark_results)
    p4 = plot(bench_df.α, bench_df.time,
    seriestype=:scatter,
    label="Время вычисления",
    xlabel="Скорость роста, α",
    ylabel="Время вычисления, сек",
    title="Зависимость времени вычисления от α",
    markersize=8,
    markercolor=:green,
    legend=:topleft
)
```

Сохраним график в папку plots

```
savefig(plotsdir(script_name, "computation_time_vs_alpha.png"))
```

5.11 Сохранение всех результатов

НОВАЯ СЕКЦИЯ: Сохранение сводных данных для последующего анализа

```
@save datadir(script_name, "all_results.jld2") base_params param_grid all_param
@save datadir(script_name, "all_plots.jld2") p1 p2 p3 p4

println("\n" * "="^60)
println("ЛАБОРАТОРНАЯ РАБОТА ЗАВЕРШЕНА")
println("="^60)
println("\nРезультаты сохранены в:")
println(" • data/$(script_name)/single/ - базовый эксперимент")
println(" • data/$(script_name)/parametric_scan/ - параметрическое сканирование
println(" • data/$(script_name)/all_results.jld2 - сводные данные")
println(" • plots/$(script_name)/ - все графики")
println(" • data/$(script_name)/all_plots.jld2 - объекты графиков")
println("\nДля анализа результатов используйте:")
println(" using JLD2, DataFrames")
println(" @load \"data/$(script_name)/all_results.jld2\"")
println(" println(results_df)")
```

6 Выводы

В ходе лабораторной работы было создано структурированное рабочее пространство для курса «Математическое моделирование» с использованием соглашения Denote и инструментария DrWatson. Выполнена реализация классической модели экспоненциального роста на языке Julia. Применён подход литературного программирования: исходный код снабжён подробными комментариями в формате Markdown, что позволило автоматически генерировать чистый скрипт, интерактивный Jupyter notebook и научный отчёт в формате Quarto. Проведено параметрическое исследование влияния скорости роста на динамику системы; результаты визуализированы и сохранены в структурированном виде. Все сгенерированные документы успешно интегрированы в итоговый отчёт. Таким образом, освоены современные методы организации воспроизводимых вычислительных проектов, которые могут быть применены при решении более сложных задач математического моделирования.

Список литературы

1. Knuth D. E. Literate Programming // The Computer Journal. — 1984. — Vol. 27, no. 2. — P. 97–111.
2. Schulte E. et al. A Multi-Language Computing Environment for Literate Programming and Reproducible Research // Journal of Statistical Software. — 2012. — Vol. 46, no. 3.
3. JuliaDynamics. DrWatson.jl documentation [Электронный ресурс]. — Режим доступа: https://juliadynamics.github.io/DrWatson.jl (дата обращения: 21.02.2026). ::: {#refs} :::